

Chap 4:

La conception

(1)

Analyse

Conception

(part entre l'analyse des besoins et l'implémentation)

Quoi faire?

Comment faire?

→ doit apporter une solution valide

Processus créatif

→ architecture (logicielle et physique)
→ description des modules et interfaces, données.

→ Activité itérative incrémentale qui transforme les besoins vers un produit final.

*) La conception est un processus de résolution des prob dont l'objectif est d'identifier la meilleure façon:
no d'implémenter les besoins fonction d'un syst
no tout en respectant les contraintes imposés par les bes. non fonction.

no en adhérant à des principes de base menant à un logiciel de qualité.

*) Résultat de cette phase = conception ou "Design"

*) Une bonne conception contribue à la fiabilité, maintenabilité, correctibilité et évolutivité du logiciel. (efficacité maintenabilité)

*) Différentes sol de conception ⇒ Optims de conception.

*) L'espace de design l'espace engendré par l'ensemble des solutions possibles pour un design.

Besoins

Conception globale

Processus en n étapes:

chacune étape raffine et décompose en sous-modules les modules définies à l'étape précédente.

Conception haut niveau

Conception bas niveau

Implémentation

La conception passe par deux étapes:

Conception architecturale

Conception détaillée.

Conception architecturale

= conception de haut niveau = conception globale

→ Première étape qui consiste à définir les fct de l'elt d'un syst et leurs relations fonctionnelles.

Obj: Structuration et organisation générale du syst à concevoir

Définir les elt principaux du syst les relations entre eux, les contraintes à respecter, les motifs et la logique de cette composition

Diviser le syst en sous-syst et composants.

Les interfaces et interconnexion et

Conception détaillée

= conception de bas niveau

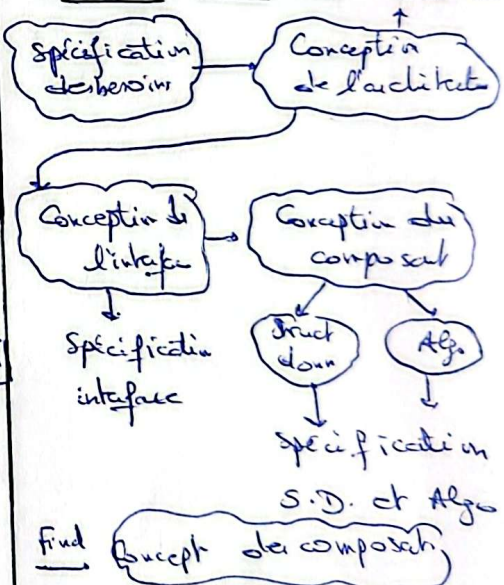
→ Détailler les résultats de l'analyse fonctionnelle jusqu'à un niveau suffisant pour permettre le codage dans un langage de prog choisi.

→ Définir un design logiciel respectant le plan de la conception architecturale

Obj:

- *) Description précise de chaque module
- *) Alg - mise en œuvre
- *) Traitements effectués en cas d'erreur
- *) Conception des données
- *) Conception des algo (efficacité)
- *) Conception des protocoles (messages et règles utilisés dans la conversation)

Processus de conception



- services affectés
- les détails des algo
- Découper les syst en plus composants.

Principes des bases de concept

- 1) Ne pas réinventer la roue (exploiter les sol existants)
- 2) Uniforme et interprétable (règles pré-établies définies pour toute l'équipe)

- ③ Structure facilitant l'évolution (Tracabilité entre élts.)
- ④ La conception n'est pas le codage et vice versa
- ⑤ Doit miser sur la qualité
- ⑥ Doit être revue pour minimiser les erreurs sémantiques.

(faire attention aux omissions, ambiguïtés, et inconsistances)

* Les Principes de Conception :

- Abstraction ^{réduire le système à son essence} aussi élevée que possible ^{implémentation}
- Encapsulation ^{Associer à une vue interne et externe (interface : ce qu'on doit faire)}
- Anticipation des changements (maintenabilité)
- Réutilisabilité ^{↑ que possible}
- Réutilisation ^{Réutiliser les composants (travaux profils de l'effort)}
- Forte cohésion à l'intérieur du module
- Faible couplage entre les modules

* Nodularité

Module : composant d'une app, contenant des def de données et/ou de type de données et/ou des fcts et constituant un tout cohérent. (fournisseur de ressources ou de services.)

Il UTILISE (nécessite la présence de)

CONTIENT (paquetage)

* Abstraction

saisir l'essence du syst sans avoir à en connaître les détails de son implémentation.

- En O.O, les classes sont des abstractions des données contenant des abstr. de procédures.
- visibilité privée → ↑ Abs
- visibilité publique → ↑ Abs
- méthodes ayant peu de param
- création de super-classes et d'interfaces.

* Anticipation des Changements

- perfectement perfective
- imposés par les nouvelles exigences du client ou utilisateur
- adaptations adaptative
- imposés par la modification de l'env matériel, social...

Type de change,

→ Algor / données / principes / env social / processus de dev

→ Pas de constat numérique ad hoc (blind coding).

* L'architecte en chef

Dans une grande entreprise, ce rôle est rempli par un responsable de dev harmoniseur de la bibliothèque et de la conservation de l'interprète de son architecture.

* Aucun logiciel de grande taille n'est développé depuis zéro aujourd'hui.

Utilisation de:

- outils de code
- Structures + fonctions / Classes + méthodes
- Bibliothèques
- Cadriciels: cadre d'application (frameworks)

Bibliothèque

logicielle: ensemble de fcts utilitaires, regroupées et mises à disposition afin de pouvoir être utilisées sans avoir à les réécrire.

Les biblio offrent une interface de prog (API: Application Programming Interface) permettant aux programmeurs de choisir des fcts.

Les API se présentent comme une liste des noms des fcts et/ou classes disponibles, avec une documentation sur les param à leur fournir et sur les résultats retournés.

Cadre d'application (framework)

un espace de travail modulaire. C'est un ensemble de biblio d'outils et de conventions permettant de dev du prog.

l'infrastructure système

Intégration intermédiaire (middleware)

orienté système de gestion de contenu

* Bibliothèques vs Cadriciels

Une bibliothèque est limitée à l'ensemble des fonctions du système, par contre un cadriciel peut être employé par extension pour inclure également l'architecture logicielle préconisée pour cette bibliothèque.

→ Une bibliothèque s'utilise, un framework s'étend ou se paramètre

→ avec une bibliothèque, le code d'une nouvelle application invoque le code de la bibliothèque *code appelé biblio.*

→ le code d'un framework appelle le code de la nouvelle application : Inversion de contrôle.

Principe de Hollywood : le code du framework invoque les parties de code représentant la nouvelle application en un certain nombre d'endroits prédéfinies nommés.

*) Une bonne conception doit être

on peut le dire

→ maintenable

→ Conception de qualité offrant une forte cohésion et un bon couplage.

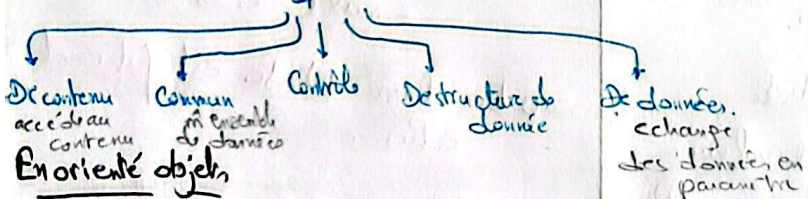
* Faible couplage : (moins de dépendance entre modules)

→ les liens de dépendance entre modules sont peu nombreux.

*) Il assure : → un bon découpage de système

→ une facilité de maintenance.

Types



En orienté objet

→ Couplage entre composants : une classe utilise en tant qu'attribut une autre classe

→ Couplage d'héritage : une des deux classes est directement ou indirectement une sous-classe de l'autre

→ Couplage d'interaction : une classe invoque une ou plusieurs méthodes de l'autre.

Remarque

L'encapsulation des données : private réduit le couplage.

Comment mesurer le couplage ?

CBO : métrique pour l'orienté objet

↳ coupling between object classes

= (Les relations avec les autres classes)

*) Forte cohésion

Mesure de la force des liens entre les éléments fonctionnels à l'intérieur d'un module

Elle assure :

- Bon découpage du système
- Facilité de maintenance
- Faible couplage

Les Types

Facile à maintenir

Fonctionnelle
Séquentielle
Communicationnelle

Difficile à maintenir

Procédurale
Temporelle
Logique
Décoïncidence

Forte cohésion

Faible cohésion

Approche procédurale

En orienté objet

→ Cohésion de méthode :

- " de classe
- " d'héritage

→ Les Types de la plus faible à la plus forte.

*) Séparable :

Les obj de classe représentent plusieurs abstractions ≠ qui n'ont aucun lien entre elles.

*) Cohésion multi-faibles

La classe représente plusieurs abstractions différentes mais liées entre elles.

*) Cohésion non déléguée

La classe ne contient pas plusieurs abstractions mais (au moins) une méthode utilise une variable d'instance pour décrire seulement un composant, et pas l'abstraction globale.

*) Cohésion cachée

La classe contient

qui pourrait être une nouvelle classe indépendante

1) Cohésion modérée (+ bonne)

La classe présente un seul concept sémantique
Aucune méthode ne devrait être déléguée ailleurs
Il n'y a pas d'abstraction cachée.

Comment mesurer la cohésion en orienté objet?

LCOM (lack of cohesion in methods)

* Méthode 1,

1) I_i = ensemble des variables d'instances utilisées par la méthode I

2) P = ensemble des paires (I_i, I_j) ayant une intersection vide

3) Φ = ensemble des paires (I_i, I_j) ayant une intersection non vide.

$$LCOM = \max(|P| - |\Phi|, 0)$$

* Méthode 2

LCOM = graphe bipartite

1) Un premier ensemble de nœuds correspond aux différents attributs et un second ensemble aux différentes méthodes.

2) Un attribut est lié à une fonction si celle-ci y accède ou en modifie la valeur.

Φ = nb d'arcs les deux parties du graphe.

P = nb d'attributs * nb-méthodes - Φ

$$LCOM = \max(|P| - |\Phi|, 0)$$

Exemples

```
public class Node {
```

```
    private int id;
```

```
    private float weightN;
```

```
    public Node(int id, float weightN) {
```

```
        this.id = id;
```

```
        this.weightN = weightN; }
```

```
    public int getId() { return this.id; }
```

```
    public float getWeightN() { return this.weightN; } }
```

Méthode I,

$I_{Node} = \{id, weightN\}$

$I_{getId} = \{id\}$

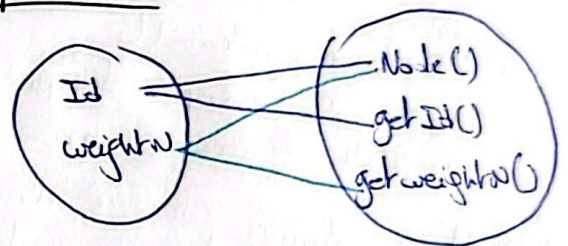
$I_{getWeightN} = \{weightN\}$

$P = \{I_{getId}, I_{getWeightN}\}$

$\Phi = \{(I_{Node}, I_{getId}), (I_{Node}, I_{getWeightN})\}$

$$\Rightarrow LCOM = \max(1-2, 0) \\ = \max(-1, 0) = 0$$

Méthode II,



Nb-attributs = 2

Nb-méthodes = 3

$\Phi = 4$

$P = 2 * 3 - 4 = 2$

$$LCOM = \max(2-4, 0) \\ = \max(-2, 0) = 0$$

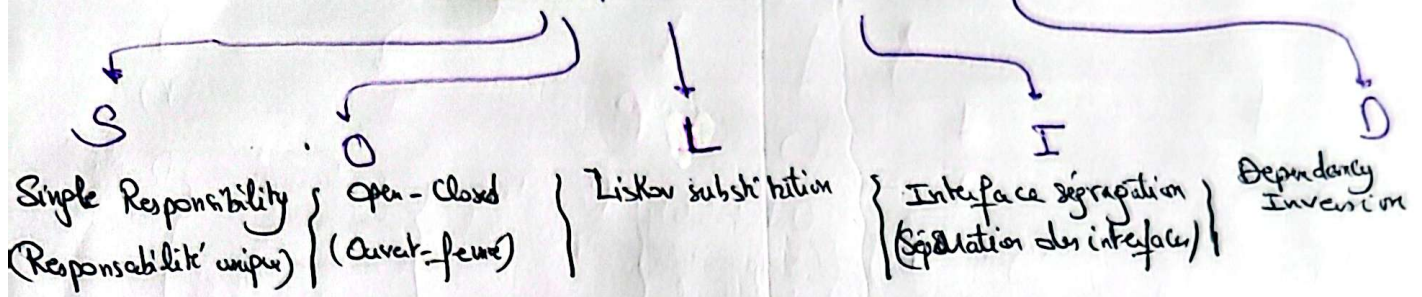
$\Rightarrow$ La classe Node est faiblement cohésive

$LCOM = 0$: cohésion parfaite

$LCOM > 0$: problème : classe n'est pas de relation.

Les Principes SOLID:

3



S → une classe doit avoir une et une seule raison de changer
(Kol class responsible filaja mch wahda responsible 3al Kol)

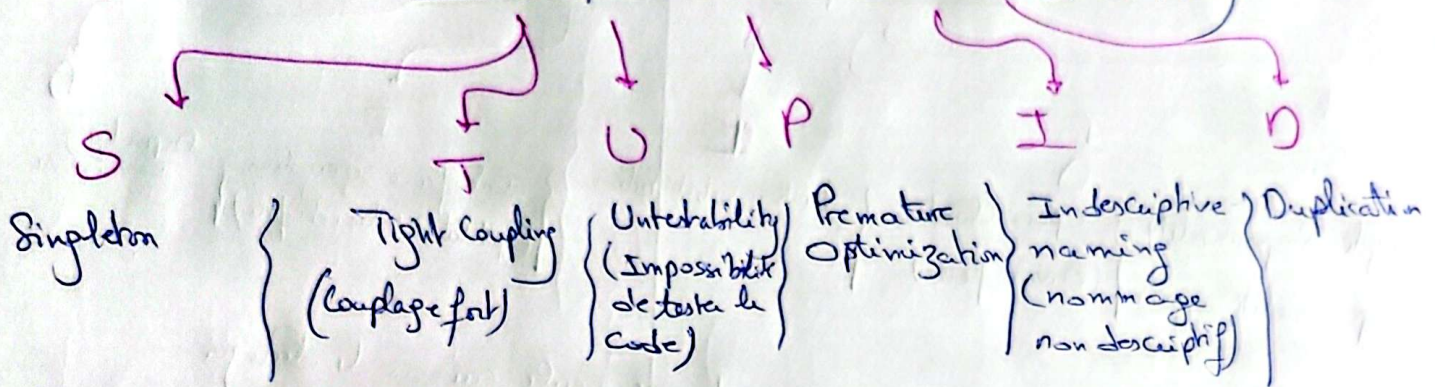
O → rendre les modules ouverts à l'extension ~~et~~ fermés aux modifications
(n'ajouter des nouvelles fonctionnalités menhir ma nbadou lcode le 3dim)

L → Les classes dérivées doivent être substituables à leurs classes de base
(Si S est une sous-classe de T, alors on doit pouvoir utiliser un objet de type S comme un obj de type T sans que ça provoque d'erreur ou de comportements inattendus)

I → principe de responsabilité unique pour les interfaces
→ Il vaut mieux avoir deux interfaces présentant un nombre limité de méthodes à implémenter, plutôt qu'une seule interface disposant de nombreuses responsabilités.

D → Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau. Les deux doivent dépendre d'abstractions (interfaces ou classes abstraites)

Les Pratiques STUPID: ^{dans} (Conception de codes difficiles à gérer et à tester)



S → Un singleton veille à ce qu'il y ait une seule instance d'une classe soit créée, ce qui peut sembler judicieux. Mais, il devient tentant de recourir à un singleton pour tout faire.

T → il faut réduire le couplage fort en codant des interfaces plutôt que des implémentations.

U → Le couplage fort est la raison principale qui rend la classe difficile ou impossible à tester.

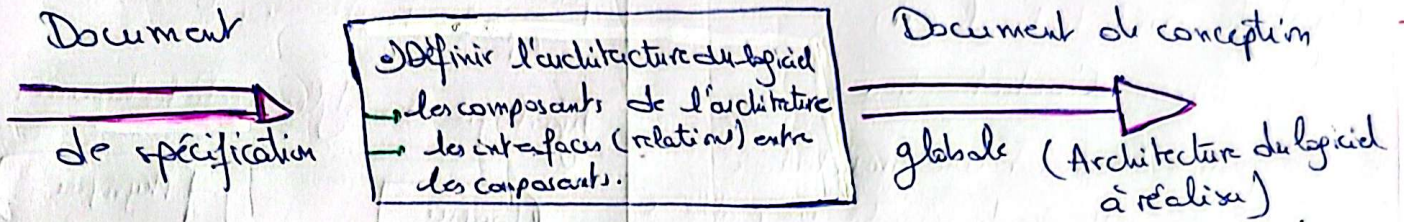
P → (Optimisation prématurée)

il ne faut pas exagérer dans l'optimisation (rend le plus complexe illisible et difficile à modifier).

I → utilise inconsciemment, aléatoirement et intuitivement un nommage non descriptif.

D → il faut éviter de Copier / Coller / Modifier.

* Conception architecturale (4)



- *) L'architecture logicielle définit la structure d'un logiciel qui comprend les éléments logiciels, leurs propriétés visibles et leurs relations.
- *) L'architecture informatique définit la structuration d'un système informatique en termes de composants et d'organisation de ses fonctions.
- *) La déf de l'architecture logicielle consiste à :
 - Décrire l'organisation générale d'un système et sa décomposition en sous-systèmes ou composants.
 - Déterminer les interfaces entre les sous-systèmes.
 - Décrire les interactions et le flux de contrôle entre les sous-systèmes.
 - Décrire également les composants utilisés pour implémenter les fonctionnalités de sous-systèmes.
 - *) Les propriétés de ces composants
 - *) Leur contenu
 - *) Les machines ou dispositifs matériels sur lesquels ces modules sont déployés.

Structuration du système

- ↳ Découpage logique → Vue logique (découpage en modules)
(Vue en couches = layer view)
- ↳ Découpage physique → Vue physique (déploiement physique prenant en compte)
(Vue en niveaux = tier view)

*) Le choix de l'architecture dépend des exigences fonctionnelles et non fonctionnelles du logiciel.

Styles architecturaux

Un style architectural (a)

Patron d'architecture :

est un patron décrivant une architecture logicielle permettant de résoudre un problème particulière.

Définitions

- 1) Un ensemble de composants et de connecteurs (type)
- 2) Les règles de configuration des composants et des connecteurs (topologie)
- 3) Une spécification du comportement du patron.
- 4) Des exemples de systèmes construits selon ce patron.

*) Un style architectural est un **patron récurrent** (commun) d'organisation / architecture du système.

Un style architectural est déterminé par :

→ Un ensemble de **types** de composants (dépt de données, processus, procédure)

→ Une structuration **topologique** des composants indiquant leurs relations durant l'exécution.

→ Un ensemble de **contraintes sémantiques** sur les composants et leurs interrelations.

→ Un ensemble de **connecteurs** (appel de procédure) pour médier la communication et la coordination entre les composants.

Patron d'architecture - Patron de conception :

Patron d'architecture :

- se situe à un niveau plus élevé d'architecture
- Apporte des solutions sur la manière de concevoir l'organisation à grande échelle d'un logiciel en faisant abstraction des détails.
- Comporte des guides de bonnes pratiques et des règles générales qui ne peuvent pas être directement traduits en code source.

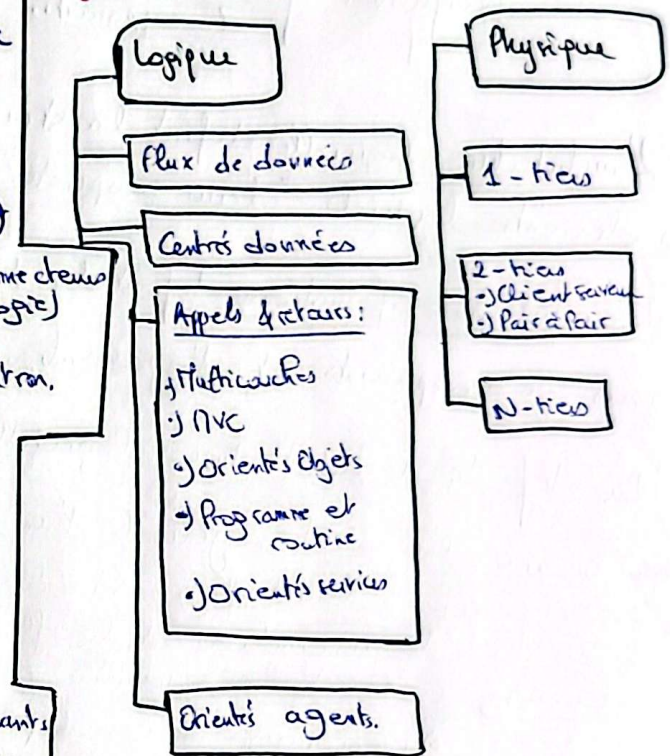
Patron de conception

proche des préoccupations locales liées à l'implémentation du code.

- au niveau de la conception détaillée
- suggère une manière d'organiser les classes
- décrit une organisation de classes fréquemment utilisée

pour résoudre un problème récurrent.
→ parle d'instances, de rôles et de collaborations

Typologie des styles architecturaux



chacun style

war bou

Style architecturaux Logiques

(5)

Flots de données

Pipeline (pipes and filters), Batch séquentiel

→ Convient aux systèmes de traitement et de transformation de données.

Composant (filtre)

Connecteur (pipe)

traite l'information

→ Reçoit ses données d'un ou plusieurs canaux d'entrée
→ Effectue la transformation / traitement des données et envoie des données de sortie produites sur un ou plusieurs canaux.

→ Fonctionnement en concurrence :
Chaque filtre traite les données au fur et mesure qu'ils les reçoit.

Canaux par lesquels transite l'information

→ Unidirectionnel au travers duquel circule un flot de données (stream)

→ Synchronisation et utilisation d'un tampon pour garantir le bon fonctionnement entre filtre producteur et filtre consommateur.

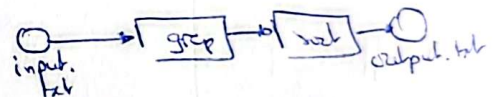
→ La configuration détermine l'ordre des traitements

Pipeline (pipes & filters) : architecture composée de "filtres" connectés par des "canaux de transfert".

Batch : une variante de l'architecture pipeline où les filtres sont en séquence.

→ Inspirés des shells de Unix

cat input.txt | grep "text" | sort > output.txt



→ C'est un modèle strictement dirigé par les données.

(+) Diviser pour régner : filtres peuvent être conçus séparément

→ Cohésion forte

→ Couplage faible : les filtres n'ont qu'une entrée et une sortie en général

→ Abstraction : filtres cachent bien leurs détails internes.

→ Réutilisabilité et réutilisation : filtres peuvent être réutilisés dans d'autres contextes nouveaux

(-) Les performances dépendent des pipes

→ Difficulté de supporter les systèmes interactifs.

Style centré sur les données

→ Un composant central (SG-BD, Datawarehouse, Blackboard) gère toutes les opérations sur les données

- Conservation
- Ajout / retrait
- Mise à jour
- Synchronisation.

→ Les composants périphériques (clients, serveurs) interagissent via ce composant central : serveur de données.

Centre de données

- passif : simple dépôt de données
→ Les clients lisent et écrivent les données mais le centre ne fait rien activement.

→ actif : (Blackboard / Tableau noir)

Mémoire de travail partagée et accessible par plusieurs clients.

Permet

- coordination des transferts d'informations entre clients.
- Communication de résultats intermédiaires.

*) Si c'est l'état du composant central qui provoque l'exécution de processus
⇒ architecture à tableau noir (Blackboard)

*) Si ce sont les modifications apportées au composant central qui provoque l'exécution des processus ⇒ base de données distribuée.

Dépôt de données (Repository)

Pour partager les données :

- ↳ Concevoir les données dans un dépôt central auquel peuvent accéder tous les sous-systèmes.
- ↳ Maintenir dans chaque sous-système un dépôt et passer explicitement les données d'un s.-système à un autre.

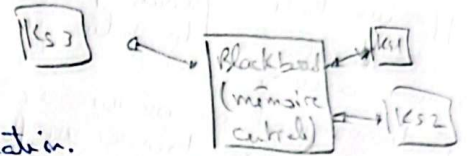
*) On utilise un **dépôt commun** lorsqu'on a un grand volume de données à partager.

Tableau noir :

*) dépôt central de données

*) des sources de connaissance (KS) -

les composants de traitement qui dépendent de l'application.



↳ **En pratique**

→ Besoin d'un module de contrôle élaboré

→ Associer les évts du Blackboard aux KS

→ Choisir des prochaines KS qui modifient le contenu du tableau

⊕ Efficace pour partager de grands volumes de données ou les applications impliquant des tâches complexes et nombreuses sur les données.

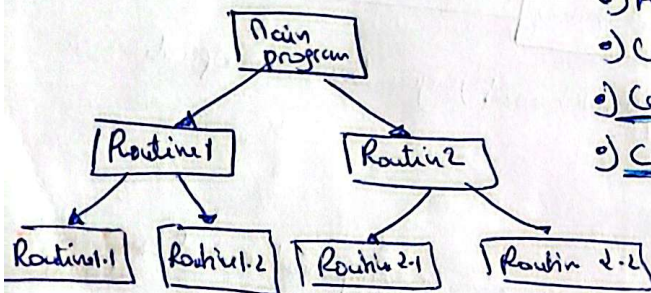
⊖ Le dépôt de données peut constituer un goulot d'étranglement, de point de vue de la performance due au partage (trop d'accès simultanés).

Styles appels et retours

→ décomposer le système en une hiérarchie de contrôle. (décomposer une fonctionnalité en sous-fonctionnalités) (en sous-fonctionnalités fonctionnelles)

→ également appelé **style requête-réponse**. (ou décomposition fonctionnelle)

1.) Programme Principal-sous-routines :



*) Approche de contrôle centralisé

*) Contrôle descendant (top-down)

*) **Composantes** : les sous-routines

*) **Connecteurs** : les appels fonctionnels.

2.) Architecture Orientée Objets :

→ structuration du système en un ensemble de modules faiblement couplés.

↳ **Composantes** : paquets ou composants **Connecteurs** : les interfaces bien définies des composants

*) Pour l'implantation, on ajoute un modèle de contrôle : Diagramme de séquence ou de collaboration qui permet de coordonner les opérations des objets.

(+) Faible couplage

- Les objets représentent des entités du domaine d'application.
- Représente les fonctionnalités verticales du système $\Rightarrow$ aspects métier.

(-) Les entités complexes parfois difficile à représenter comme des objets.

- Éléments de code impossible à centraliser
- Héritage ne permet pas d'éviter complètement la duplication de code.
- Les fonctionnalités horizontales ou traverses $\Rightarrow$ aspects techniques de l'application $\rightarrow$ s'étalent sur un grand nombre de modules.

3.) Architecture Multicouche

- Composants: les couches: chaque couche réalise un service
- Connecteurs: dépendent du protocole d'interaction souhaité entre couches
 - Système ouvert: toutes les couches ont accès à toutes les autres. Les connecteurs peuvent relier deux couches quelconques.
 - Système fermé: une couche n'a accès qu'aux couches adjacentes. Les connecteurs ne relient que les couches adjacentes. (Exemple: Modèle OSI (mbrs réels))

- Chaque couche est un composant avec une interface bien définie utilisée par la couche juste au-dessus.
- La couche supérieure (extérieure) voit la couche inférieure (interne) comme un ensemble de services offerts.
- Les couches les plus basses offrent les services généraux (accès à la Base de données...)
- Il est important d'avoir une couche séparée par l'UI
 - Les couches juste au-dessous de l'UI offrent des fcts applicatives définies par le cas d'utilisation.

- (+) Diviser pour régner
 - Cohésion / Couplage / Abstraction
 - Réutilisabilité et Réutilisation

- (-) La bonne décomposition en couches.

*) Variante 1: Modèle trois couches:

- La couche IHM ou présentation: (Interface Homme Machine)
gère les interactions H/M
- La couche logique applicative:
Services métier de l'application et contrôle effectués au niveau du dialogue avec l'IHM
- La couche données: Gère le stockage des données et l'accès à ces données.

*) Variante 2: Modèle cinq couches:

- *) Présentation: assure l'affichage des IHM
- *) Contrôleur / Coordination: contrôle la cinématique des écueils.
- *) Service: implémente la logique métier.
- *) Domaine: gestion des données métier.
- *) Assistance: fournit des services de stockage des données et de mapping entre les différentes formes des objets métier du domaine.

MVC: (Modèle - Vue - Contrôleur)

- *) Modèle: (Base de données) représente l'ensemble des composants qui sont chargés de réaliser des appels à la couche vue et de mettre les résultats de l'appel à la disposition de la vue
- *) Vue: représente l'interface utilisateur;
- *) Contrôleur: gère la synchronisation entre la vue et le modèle.

- ⊕ Divisé pour régner
cohésion (+ si les couches vue et contrôle étaient de l'interface utilisateur)
Couplage (le nb de canaux de communication entre les composants déjà existants)
Flexibilité (facilité de changer l'interface utilisateur)
Testabilité (il est possible de tester l'application indépendamment de l'interface)
- ⇒ goulot d'étranglement possible

4*) Architecture Orientée Services: → reposant sur les standards de l'Internet permettant à des applications de communiquer sans préoccupation des technologies d'implantations utilisées de part et d'autre. (Interopérabilité)

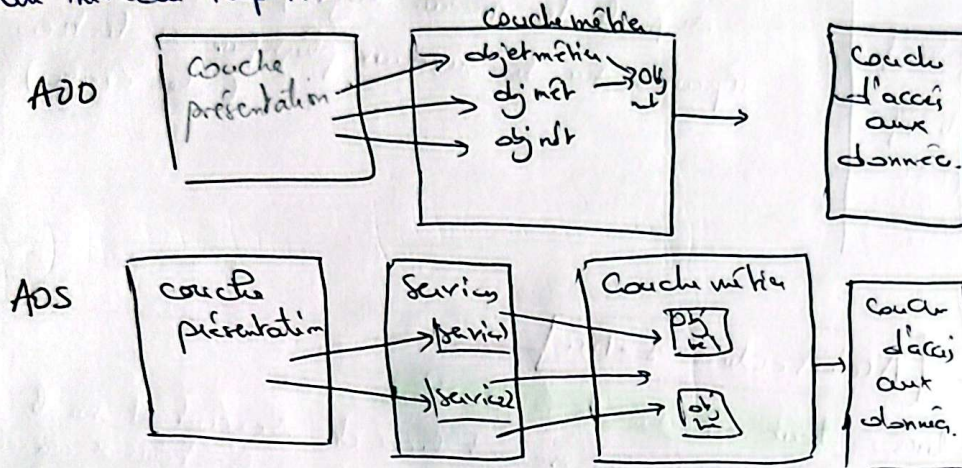
- *) SOA est basée sur des services: → faiblement couplés
→ indépendants de protocoles, basés sur les standards et distribués
→ autonomes
→ composables: crée de nouveaux services à partir d'autres services.
→ réutilisable
→ basés sur des standards

Un service SOA dialogue avec ses consommateurs sous une forme standardisée, tant au plan technique que sur le plan métier.

- Les composants de SOA → le consommateur utilise le service
→ le fournisseur assure le service
→ le registre fait le lien entre le fournisseur et le consommateur.

•) SDA constitue un style architectural basé sur le principe de séparation de l'activité métier en une série de services.

→ Ces services peuvent être assemblés et liés entre eux selon le principe de couplage lâche pour exécuter l'appli désirée. (ils sont définis à un niveau supérieur de la traditionnelle approche composable)



⊖ Nécessité d'apprendre de nouvelles technologies (with)

Systemes Orientés Agents

•) C'est un paradigme où les composants logiciels ne sont plus passifs, mais actifs et autonomes.

≠ POO: objet est passif offre des services et utilise d'autres objets

-) L'agent logiciel est: autonome / actif et doté de sa propre capacité d'exécution
-) Il dialogue, négocie et échange des infos
-) Décide dynamiquement avec quels agents interagir, selon ses besoins et la disponibilité des autres.

Système intelligent = ensemble d'unités computationnelles (agents) → agents cognitifs

(iklanman)

et i Saurau...

-) Agent:
 -) capacité à réagir et à gérer des événements
 -) caractérisé par un état
 -) possède une capacité d'expertise (rôle)
 -) relation utilisateur ⇒ interacteur
 -) capable de se réagir...

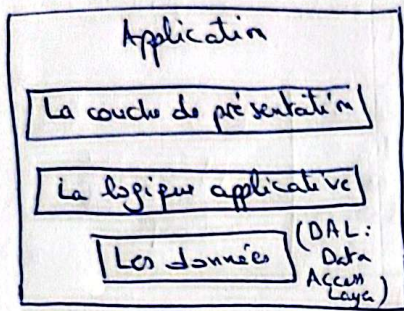
Correspondance avec l'approche des obj

-) catégorie d'agents (réactifs) → classe
-) événement → méthode
-) encapsulation: l'agent (l'objet) est seul à modifier directement son état
-) mécanisme de sous-classe → modifiabilité

•) Modularité / parallélisme / modifiabilité / dialogue à plusieurs fils / mix en œuvre des collectibles.

Les styles architecturaux physiques

Niveau d'abstraction d'une application:



Toute application possède ces 3 parties :

on parle de niveau logiques pouvant être répartis sur des niveaux physiques ou des tiers (des étages)

Architecture 1-tier

ou Architecture centralisée: tout est sur la même machine

→ les 3 couches applicatives sont intimement liées et s'exécutent sur le même ordinateur.

*) Archi 1-tier dans le contexte multi-utilisateurs!

→ application sur site centrale (Mainframe) Maître / Esclave

les utilisateurs se connectent au mainframe à l'aide de terminaux passifs.

→ app répartie sur des machines indépendantes communiquant par partage de fichiers.

*) Correspond à 1 seul niveau physique où sont hébergées toutes les couches du système.

→ Le moteur de base de données est exécuté indépendamment ou chaque point client. (bon ça pour petits groupes de travail)

mais peu fiable si ça tombe en panne les données finissent dans le poubelle

Architecture 2-tiers

visé à répartir un système informatif sur des machines distantes, grâce à des matériels et des protocoles standards.

Architecture Client-serveur

Architecture Pair-à-pair.

Architecture client-serveur, mode de communication à travers un réseau entre plusieurs logiciels de deux types :

- ① client : envoie des requêtes (programme qui provoque le dialogue)
- ② serveur : attend les requêtes et y répondent.

Les serveurs ont une grande capacité

Middleware, "élément du milieu" : ensemble de couches réseau et services logiciels qui permettent le dialogue.

= API + FAP

Application Programming Interface (si standard permet la portabilité).

↳ Format and Protocol (permet de passer d'un espace d'adressage à un autre et d'une machine à une autre).

3 types de clients :

client léger

L'application fonctionne entièrement sur le serveur, le post client y accède via un navigateur web
↳ navigateur : client universel

client lourd

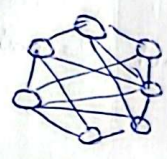
Le post client est capable d'afficher en local des IHM et d'exécuter des parties de traitements (rigide, coûteux et complexe)

client riche

comme client lourd + capable de gérer une partie de données en local.

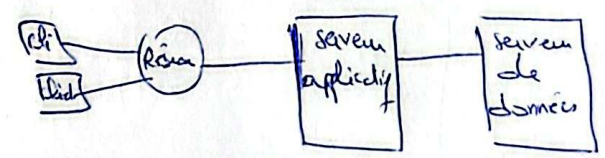
Architecture Pair à Pair : Peer to Peer : P2P :

une variante d'architecture client-serveur où chaque client est aussi un serveur
↳ On appelle **serveur** (serveur + client) un nœud (ordinateur) de cette archi qui se base sur la fusion des fct client et serveur.
Exple : système de streaming / messagerie en ligne / partage de fichiers



Architecture 3-tiers

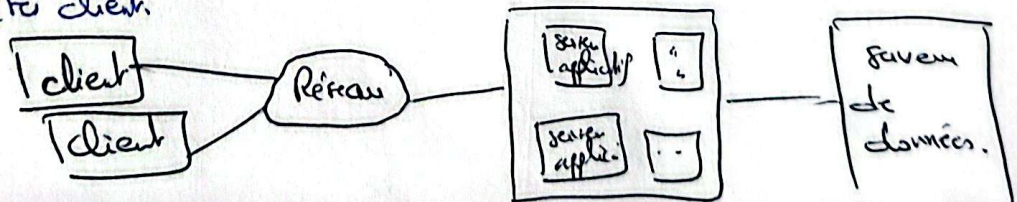
- La présentation et les traitements sont pris en charge par le client et/ou le serveur intermédiaire selon (le type du client)
- Les données sont gérées par un **serveur de données**.
- La présentation est toujours prise en compte par le post client.
- La logique applicative est prise en charge par un serveur intermédiaire : serveur applicatif



Architecture N-tiers

- ou multi-tiers
- qualifie la distribution de traitements entre de multiples serveurs d'application (qui peuvent être hétérogènes) ou la distribution des données entre plusieurs serveurs de données.
- Le serveur d'application de l'archi 3-tiers est décomposé en plusieurs applications (par exple un serveur par région)

L'avantage par rapport à l'archi 3-tiers est en terme de performance : le délai de réponse aux requêtes client.



*) Cloud Computing

- *) Technologie basée sur Internet pour fournir des services matériels et logiciels de façon évolutive
- *) Le fournisseur Cloud gère l'infrastructure IT (serveurs, stockage, réseau)
- *) L'utilisateur n'a pas à gérer la maintenance ni l'évolution des logiciels
- *) Permet de construire des architectures flexibles basées sur :
 - SOA
 - multi-couche
 - N-tier
- *) Les systèmes cloud sont souvent hétérogènes et combinent plusieurs styles architecturaux
 - *) 3 Couches :
 - SaaS (Software as a Service)
 - PaaS (Platform as a Service)
 - IaaS (Infrastructure as a Service)
 - Une structure basée sur le cloud est théoriquement infaillible
 - le cloud Computing réduit les coûts.